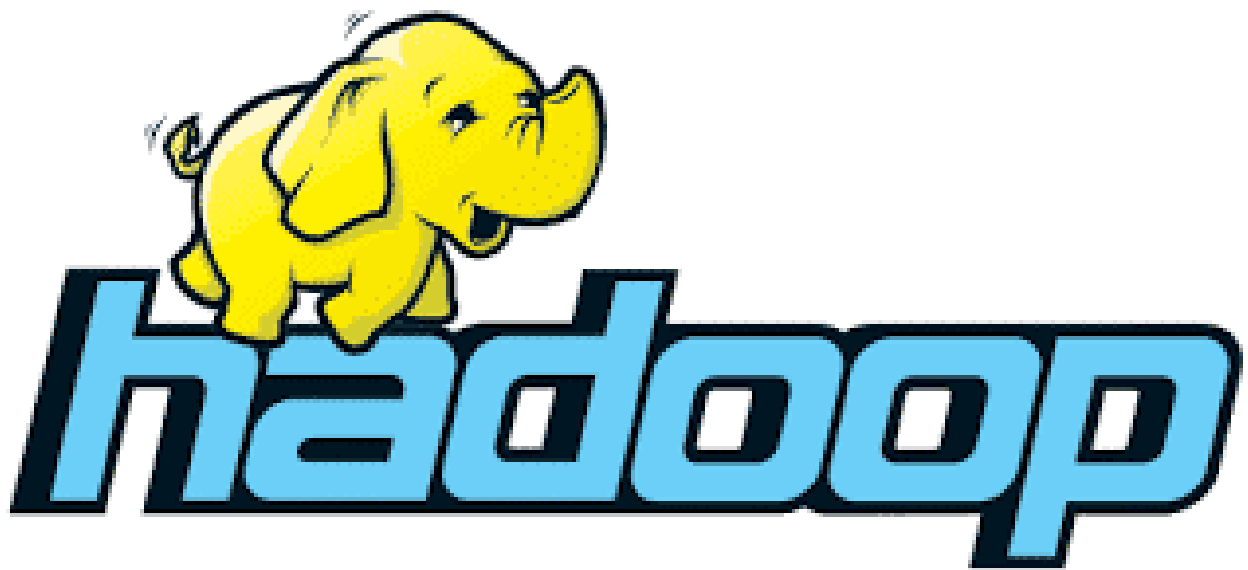


INSTALACIÓN Y PRUEBAS DE HADOOP Y HDFS EN GOOGLE COLAB



Informe de Prácticas: Instalación y Pruebas de Hadoop y HDFS en Google Colab

1. Introducción a las Tecnologías

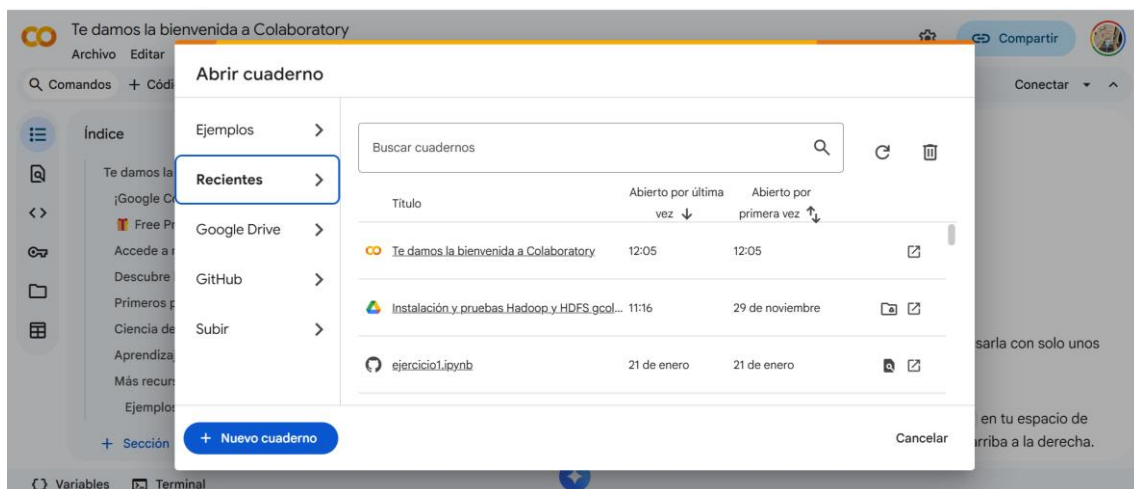
Este informe documenta la instalación, configuración y prueba del ecosistema Hadoop, incluyendo su sistema de archivos distribuido (HDFS) y su modelo de programación (MapReduce), utilizando Google Colab como entorno de ejecución.

- **Hadoop:** Es un marco de trabajo diseñado para procesar y almacenar grandes volúmenes de datos distribuidos en un clúster de ordenadores.
- **HDFS (Hadoop Distributed File System):** Es el sistema de archivos tolerante a fallos de Hadoop, optimizado para manejar esos grandes conjuntos de datos.
- **MapReduce:** Es el modelo de programación central de Hadoop, estructurado en dos fases: Map (mapear/contar) y Reduce (sumar/agrupar).

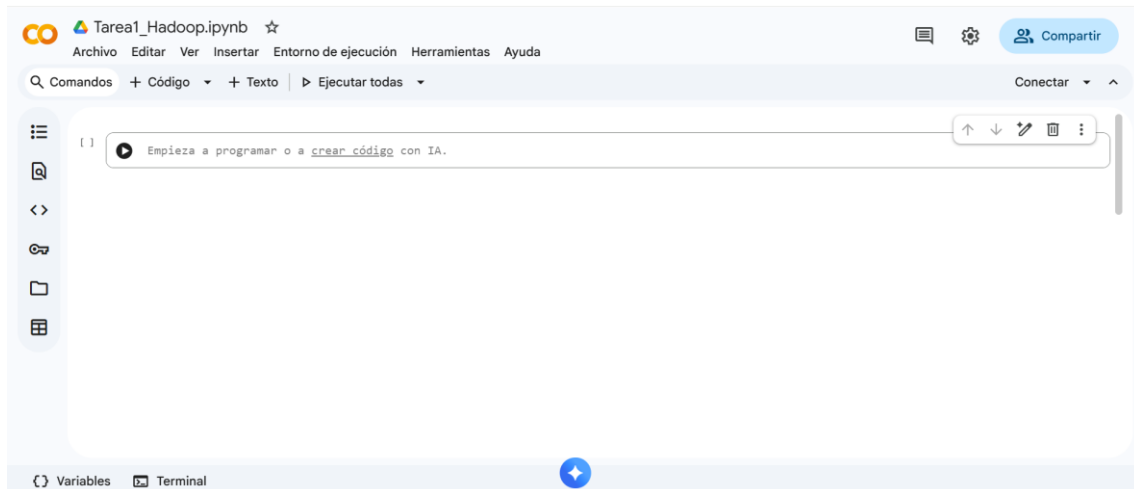
2. Instalación y Configuración del Entorno

La instalación se realizó mediante comandos de Linux en el entorno virtual de Google Colab.

Paso 1: Abrimos Google Colab



Paso 2: Creamos el Cuaderno de Instalación



Paso 3: Instalación de Hadoop

La instalación requirió la descarga del paquete de Hadoop (versión 3.3.4), su descompresión y copia se hará en el directorio local /usr/local/.

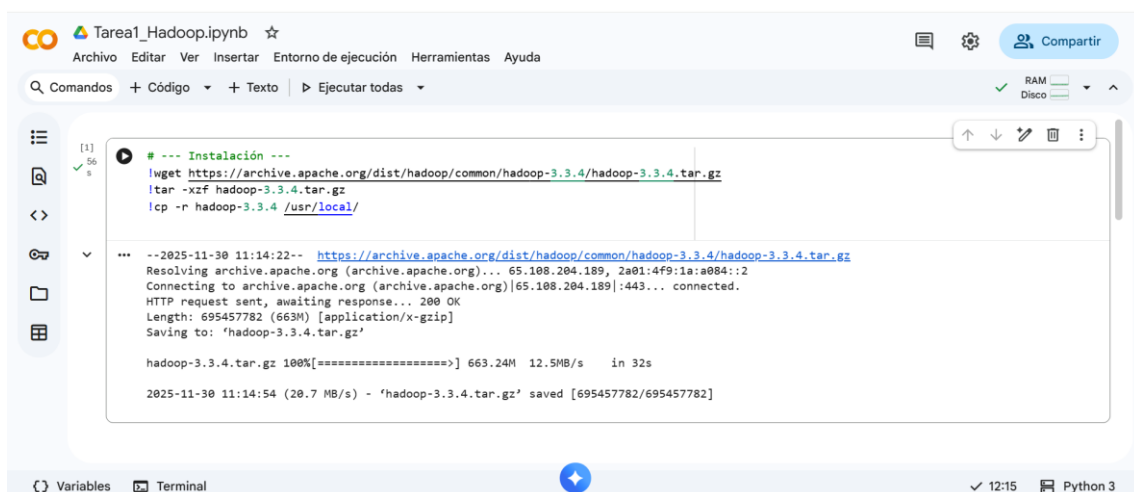
3.1. Bloque de Instalación

--- Instalación ---

!wget https://archive.apache.org/dist/hadoop/common/hadoop-3.3.4/hadoop-3.3.4.tar.gz

!tar -xzf hadoop-3.3.4.tar.gz

!cp -r hadoop-3.3.4 /usr/local/



3.2. Bloque de Configuración de Java

En una nueva celda de código, verificaremos la ruta de Java con el comando ***!readlink -f /usr/bin/java | sed "s:bin/java::"***

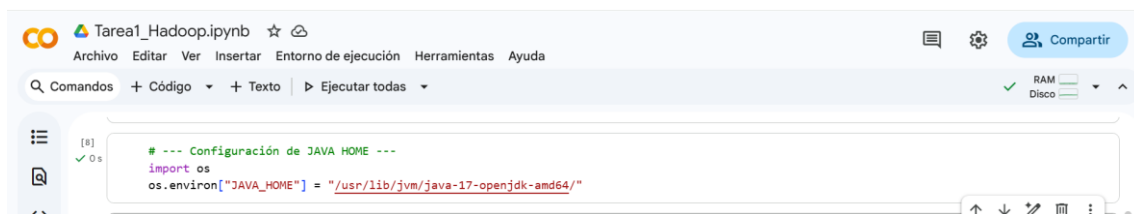
```
!readlink -f /usr/bin/java | sed "s:bin/java::"
... /usr/lib/jvm/java-17-openjdk-amd64/
```

Y posteriormente procedemos a la configuración de Java

--- Configuración de JAVA HOME ---

import os

os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-11-openjdk-amd64/"



The screenshot shows a Jupyter Notebook interface with a cell containing the following code:

```
# --- Configuración de JAVA HOME ---
import os
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-17-openjdk-amd64/"
```

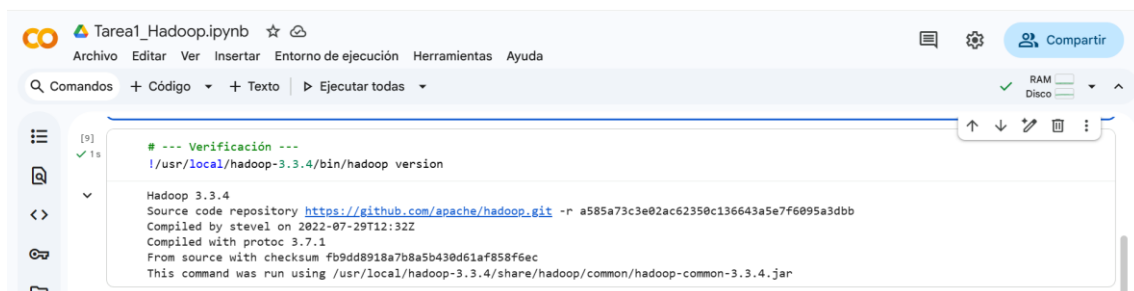
The cell is executed, and the output is empty, indicating successful execution.

3.3. Bloque de Verificación

En otra Celda, verificaremos la instalación y funcionalidad de Hadoop

--- Verificación ---

!usr/local/hadoop-3.3.4/bin/hadoop versión



The screenshot shows a Jupyter Notebook interface with a cell containing the following code:

```
# --- Verificación ---
!usr/local/hadoop-3.3.4/bin/hadoop version
```

The cell is executed, and the output is as follows:

```
Hadoop 3.3.4
Source code repository https://github.com/apache/hadoop.git -r a585a73c3e02ac62350c136643a5e7f6095a3dbb
Compiled by stevel on 2022-07-29T12:32Z
Compiled with protoc 3.7.1
From source with checksum fb9dd8918a7b8a5b430d61af858f6ec
This command was run using /usr/local/hadoop-3.3.4/share/hadoop/common/hadoop-common-3.3.4.jar
```

3. Ejercicio 1: Pruebas de HDFS

El objetivo fue practicar los comandos básicos de gestión de archivos en HDFS. Todos los comandos se ejecutaron usando el prefijo ***/usr/local/hadoop-3.3.4/bin/hdfs dfs.*** | Tarea | Comando Ejecutado | Resultado Verificado | | :--- | :--- | :--- | | Creación | ***hdfs dfs -mkdir /files*** | Directorio /files creado exitosamente. | | Carga (Put) | ***hdfs dfs -put hoy.txt /*** | Archivo local hoy.txt subido al directorio raíz. |

Copia y Muestra | `hdfs dfs -cp /hoy.txt /files/` y `hdfs dfs -cat /hoy.txt` | Contenido de hoy.txt ("Sun Nov 30 11:26:11 AM UTC 2025\nroot") mostrado correctamente. | Movimiento y Eliminación | `hdfs dfs -mv /jps.txt /files/` y `hdfs dfs -rm /hoy.txt` | Archivo hoy.txt eliminado y archivo jps.txt movido a la carpeta /files/. |

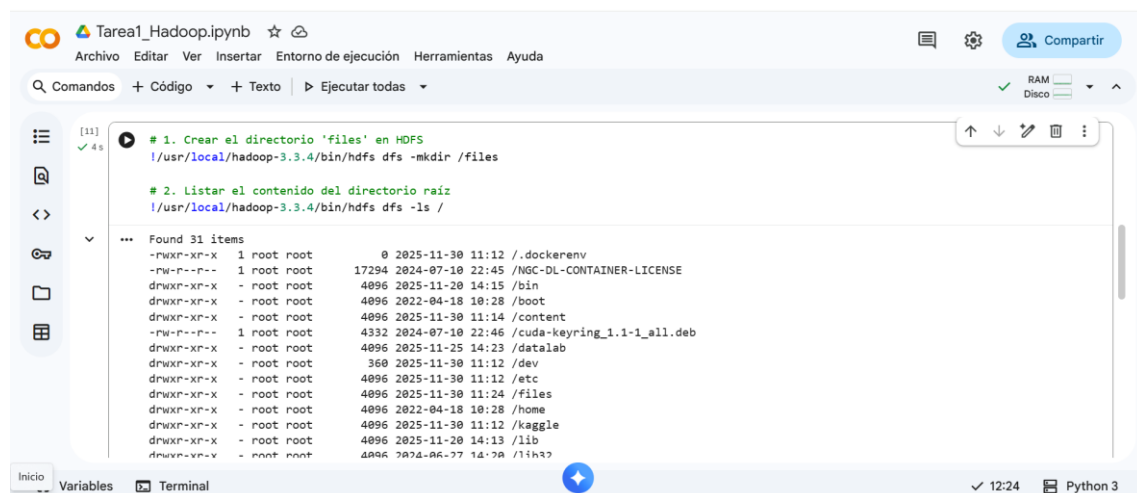
3.1. Tarea 1 y 2: Crear y Listar

1. Crear el directorio 'files' en HDFS

`!usr/local/hadoop-3.3.4/bin/hdfs dfs -mkdir /files`

2. Listar el contenido del directorio raíz

`!usr/local/hadoop-3.3.4/bin/hdfs dfs -ls /`



```
# 1. Crear el directorio 'files' en HDFS
/usr/local/hadoop-3.3.4/bin/hdfs dfs -mkdir /files

# 2. Listar el contenido del directorio raíz
/usr/local/hadoop-3.3.4/bin/hdfs dfs -ls /
```

```
Found 31 items
-rwxr-xr-x 1 root root      0 2025-11-30 11:12 /.dockerenv
-rw-r--r-- 1 root root 17294 2024-07-10 22:45 /NGC-DL-CONTAINER-LICENSE
drwxr-xr-x - root root    4096 2025-11-20 14:15 /bin
drwxr-xr-x - root root    4096 2022-04-18 10:28 /boot
drwxr-xr-x - root root    4096 2025-11-30 11:14 /content
-rw-r--r-- 1 root root 4332 2024-07-10 22:46 /cuda-keyring_1.1-1_all.deb
drwxr-xr-x - root root    4096 2025-11-25 14:23 /datalab
drwxr-xr-x - root root    360 2025-11-30 11:12 /dev
drwxr-xr-x - root root    4096 2025-11-30 11:12 /etc
drwxr-xr-x - root root    4096 2025-11-30 11:24 /files
drwxr-xr-x - root root    4096 2022-04-18 10:28 /home
drwxr-xr-x - root root    4096 2025-11-30 11:12 /kaggle
drwxr-xr-x - root root    4096 2025-11-20 14:13 /lib
drwxr-xr-x - root root    4096 2024-06-27 14:20 /lib32
```

3.2. Tarea 3 y 4: Subir Archivo y Mostrar Contenido

3. Crear el archivo 'hoy.txt' localmente y cargarlo a HDFS

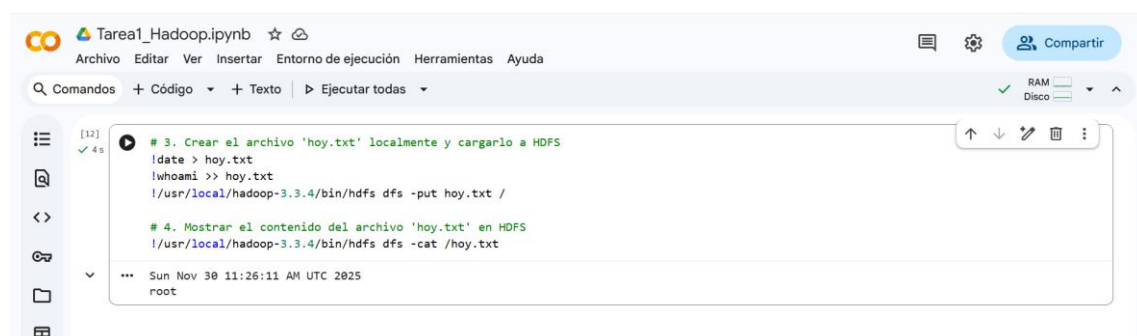
`!date > hoy.txt`

`!whoami >> hoy.txt`

`!usr/local/hadoop-3.3.4/bin/hdfs dfs -put hoy.txt /`

4. Mostrar el contenido del archivo 'hoy.txt' en HDFS

`!usr/local/hadoop-3.3.4/bin/hdfs dfs -cat /hoy.txt`



```
# 3. Crear el archivo 'hoy.txt' localmente y cargarlo a HDFS
!date > hoy.txt
!whoami >> hoy.txt
/usr/local/hadoop-3.3.4/bin/hdfs dfs -put hoy.txt /

# 4. Mostrar el contenido del archivo 'hoy.txt' en HDFS
/usr/local/hadoop-3.3.4/bin/hdfs dfs -cat /hoy.txt
```

```
Sun Nov 30 11:26:11 AM UTC 2025
root
```

3.3. Tarea 5, 6, 7, 8 y 9: Gestión de Archivos

5. Copiar 'hoy.txt' al directorio 'files'

```
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -cp /hoy.txt /files/
```

6. Crear el archivo 'jps.txt' localmente y cargarlo a HDFS

```
!jps > jps.txt
```

```
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -put jps.txt /
```

7. Mover 'jps.txt' al directorio 'files'

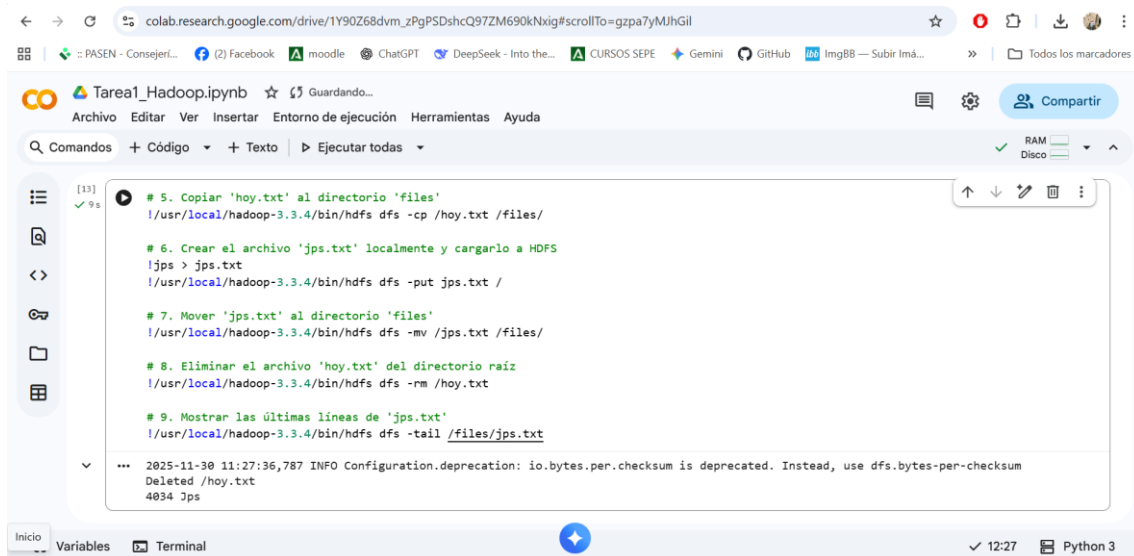
```
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -mv /jps.txt /files/
```

8. Eliminar el archivo 'hoy.txt' del directorio raíz

```
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -rm /hoy.txt
```

9. Mostrar las últimas líneas de 'jps.txt'

```
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -tail /files/jps.txt
```



The screenshot shows a Google Colab notebook titled 'Tarea1_Hadoop.ipynb'. The notebook contains a series of Hadoop commands and their outputs. The commands are: 1. Copying 'hoy.txt' to the 'files' directory. 2. Creating 'jps.txt' locally and uploading it to HDFS. 3. Moving 'jps.txt' to the 'files' directory. 4. Deleting 'hoy.txt' from the root directory. 5. Showing the last lines of 'jps.txt' in the 'files' directory. The output shows the successful execution of these commands, including a deprecation warning for the 'dfs.bytes-per-checksum' configuration.

```
[13] ✓ 9s  
# 5. Copiar 'hoy.txt' al directorio 'files'  
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -cp /hoy.txt /files/  
  
# 6. Crear el archivo 'jps.txt' localmente y cargarlo a HDFS  
!jps > jps.txt  
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -put jps.txt /  
  
# 7. Mover 'jps.txt' al directorio 'files'  
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -mv /jps.txt /files/  
  
# 8. Eliminar el archivo 'hoy.txt' del directorio raíz  
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -rm /hoy.txt  
  
# 9. Mostrar las últimas líneas de 'jps.txt'  
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -tail /files/jps.txt  
  
... 2025-11-30 11:27:36,787 INFO Configuration.deprecation: io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum  
Deleted /hoy.txt  
4034 Jps
```

4- Ejercicio 2: WordCount

Dominado la gestión de archivos con HDFS, el siguiente paso es el Ejercicio WordCount (Contador de Palabras), que es el primer trabajo de MapReduce que vamos a realizar.

Este ejercicio requiere tres pasos clave:

1. Descargar el texto de El Quijote y cargarlo a HDFS.
2. Ejecutar el programa wordcount de Hadoop.
3. Mostrar y ordenar los resultados.

Paso 1: Preparar los Datos

Debes crear una nueva Celda de Código y ejecutar los siguientes comandos para descargar el texto de prueba y colocarlo en HDFS.

1. Descargar el texto de El Quijote

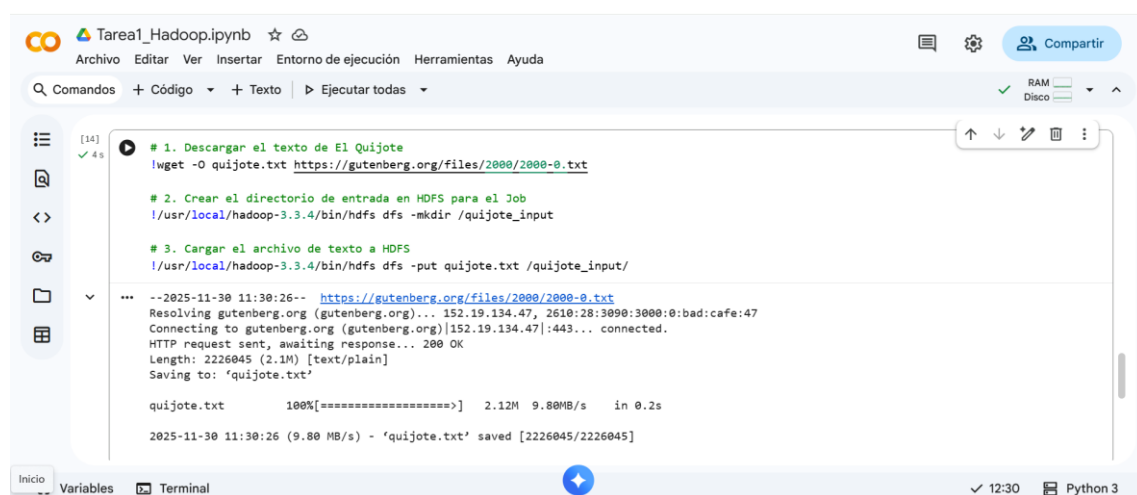
```
!wget -O quijote.txt https://gutenberg.org/files/2000/2000-0.txt
```

2. Crear el directorio de entrada en HDFS para el Job

```
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -mkdir /quijote_input
```

3. Cargar el archivo de texto a HDFS

```
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -put quijote.txt /quijote_input/
```



The screenshot shows a Jupyter Notebook titled 'Tarea1_Hadoop.ipynb'. The code cell contains three commands: 1. Downloading 'quijote.txt' from Gutenberg. 2. Creating the directory '/quijote_input' in HDFS. 3. Uploading 'quijote.txt' to HDFS. The output shows the download progress and the successful upload to HDFS.

```
# 1. Descargar el texto de El Quijote
!wget -O quijote.txt https://gutenberg.org/files/2000/2000-0.txt

# 2. Crear el directorio de entrada en HDFS para el Job
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -mkdir /quijote_input

# 3. Cargar el archivo de texto a HDFS
!/usr/local/hadoop-3.3.4/bin/hdfs dfs -put quijote.txt /quijote_input/

--2025-11-30 11:30:26-- https://gutenberg.org/files/2000/2000-0.txt
Resolving gutenberg.org (gutenberg.org)... 152.19.134.47, 2610:28:3090:0:bad:cafe:47
Connecting to gutenberg.org (gutenberg.org)|152.19.134.47|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2226045 (2.1M) [text/plain]
Saving to: 'quijote.txt'

quijote.txt      100%[=====] 2.12M  9.80MB/s   in 0.2s

2025-11-30 11:30:26 (9.80 MB/s) - 'quijote.txt' saved [2226045/2226045]
```

Paso 2: Ejecutar el Job de WordCount (MapReduce)

Creamos otra nueva Celda de Código. Este comando es el corazón del ejercicio: le dice a Hadoop que use el programa wordcount con tu archivo de entrada.

Ejecuta el trabajo MapReduce WordCount.

Entrada: /quijote_input/

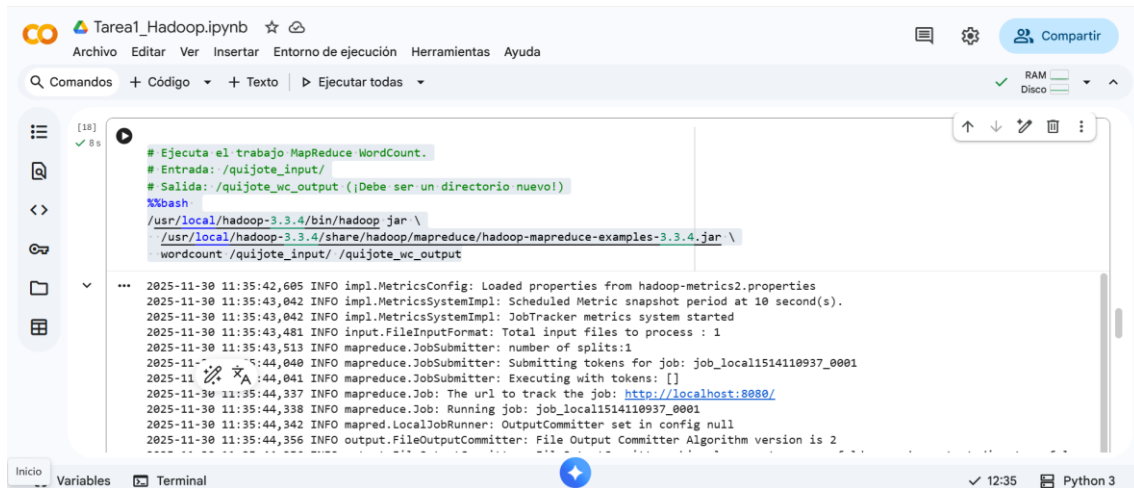
Salida: /quijote_wc_output (¡Debe ser un directorio nuevo!)

%%bash

**/usr/local/hadoop-3.3.4/bin/hadoop jar **

**/usr/local/hadoop-3.3.4/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.4.jar **

wordcount /quijote_input/ /quijote_wc_output



```
[18]
# Ejecuta el trabajo MapReduce WordCount.
# Entrada: /quijote_input/
# Salida: /quijote_wc_output (¡Debe ser un directorio nuevo!)
%%bash
/usr/local/hadoop-3.3.4/bin/hadoop jar \
    /usr/local/hadoop-3.3.4/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.4.jar \
    wordcount /quijote_input/ /quijote_wc_output

2025-11-30 11:35:42,605 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2025-11-30 11:35:43,042 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2025-11-30 11:35:43,042 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2025-11-30 11:35:43,481 INFO input.FileInputFormat: Total input files to process : 1
2025-11-30 11:35:43,513 INFO mapreduce.JobSubmitter: number of splits:1
2025-11-30 11:35:44,040 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local1514110937_0001
2025-11-30 11:35:44,041 INFO mapreduce.JobSubmitter: Executing with tokens: []
2025-11-30 11:35:44,337 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2025-11-30 11:35:44,338 INFO mapreduce.Job: Running job: job_local1514110937_0001
2025-11-30 11:35:44,342 INFO mapred.LocalJobRunner: OutputCommitter set in config null
2025-11-30 11:35:44,356 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
```

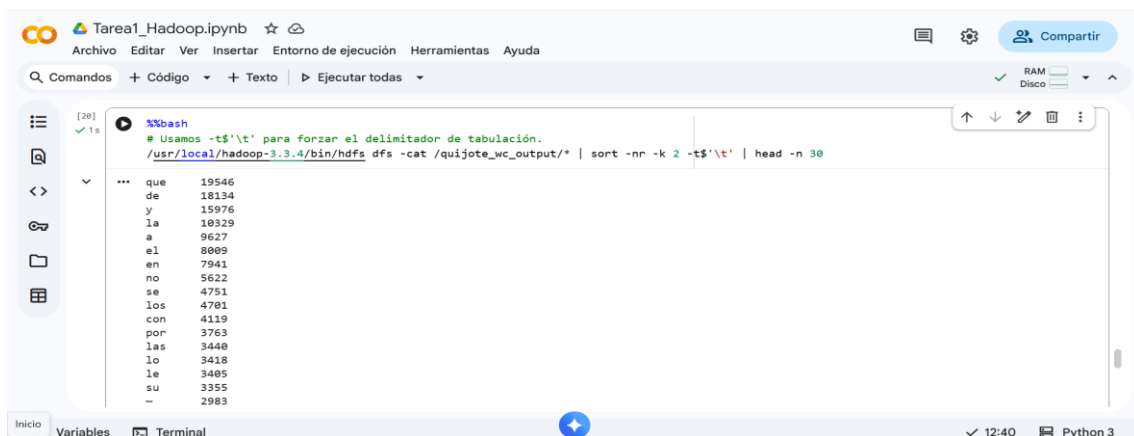
Paso 3: Mostrar y Ordenar los Resultados

Este paso lee los resultados del conteo y los ordena, mostrando las palabras más frecuentes primero. En una nueva celda de código:

%%bash

/usr/local/hadoop-3.3.4/bin/hdfs dfs -cat /quijote_wc_output/* | sort -nr -k 2 -t\$'\t'

Lee el resultado (palabra y conteo) y lo ordena numéricamente de forma inversa (de mayor a menor)



```
[20]
%%bash
# Usamos -t$'\t' para forzar el delimitador de tabulación.
/usr/local/hadoop-3.3.4/bin/hdfs dfs -cat /quijote_wc_output/* | sort -nr -k 2 -t$'\t' | head -n 30

que 19546
de 18134
y 15976
la 10329
a 9627
el 8009
en 7941
no 5622
se 4751
los 4701
con 4119
por 3763
las 3440
lo 3418
le 3405
su 3355
- 2983
```


Este resultado confirma que el proceso MapReduce se ejecutó exitosamente, contando las apariciones de cada palabra y demostrando la capacidad del sistema para procesar grandes archivos de texto.